CS 246 Watopoly — DD1 Plan

Cristophe Chen, Brian Fu, Kevin Wang

This document pairs with `uml.pdf` and explains the classes, their responsibilities, and how the project will be implemented. It also answers the design questions posed in the Watopoly project specification.

---

1. Goals & Scope

- 2–6 player, turn-based Monopoly variant ("Watopoly") with a 40-square board themed around the University of Waterloo campus.
- Text-based board display showing square names, player tokens, and improvement markers.
- Full command interpreter supporting rolling, trading, improving, mortgaging, bankruptcy, auctions, saving, and loading.
- Testing mode (`-testing`) allowing deterministic dice rolls via `roll <die1> <die2>` .
- Game state persistence: save to file and load from file (`-load <file>`).
- Robust input handling — misspelled or invalid commands should not crash the program.
- Design for extensibility: easy to add new square types, new commands, new display modes, and rule changes with minimal recompilation.

---

2. High-Level Architecture

See `uml.pdf` for the full class diagram.

- **Core Engine**
  - **Key classes:** `WatopolyGame` , `IGameContext` , `Board` , `Player` , `Dice` , `TurnContext`
  - **Purpose:** Central game loop, board state, player state, dice rolling, doubles tracking, and per-turn state. `IGameContext` is the narrow abstract interface that all Squares and Commands depend on, decoupling them from the concrete `WatopolyGame` class (Dependency Inversion Principle).
- **Squares**
  - **Key classes:** `Square` hierarchy (`AcademicBuilding` , `Residence` , `Gym` , `CollectOSAP` , `DCTimsLine` , `GoToTims` , `GooseNesting` , `TuitionSquare` , `CoopFee` , `SLCSquare` , `NeedlesHallSquare`)
  - **Purpose:** Landing logic for each of the 40 board squares. All square `landOn` methods take `IGameContext &` — they call back into the game through the abstract interface, not through a concrete class.
- **Ownership & Economy**
  - **Key classes:** `Property` (abstract), `MonopolyBlock` , `Auction` , `Trade`
  - **Purpose:** Property ownership, monopoly grouping, improvement management, auction bidding, and trade negotiation.
- **Command Layer**
  - **Key classes:** `CommandInterpreter` , `Command` hierarchy
  - **Purpose:** Parsing user input, dispatching game actions, and handling invalid input gracefully. Commands depend on `IGameContext &` . Each `Command` implements `isValidInPhase(TurnPhase)` to enforce command restrictions per turn phase.
- **Persistence**
  - **Key classes:** `SaveManager`
  - **Purpose:** Serialization and deserialization of full game state to/from the required save-file format.
- **Presentation**
  - **Key classes:** `BoardDisplay` , `DisplayObserver`
  - **Purpose:** Text rendering plus an observer abstraction to support future displays without changing game logic.
- **Randomness**
  - **Key classes:** `RandomEventGenerator`
  - **Purpose:** Centralized RNG for dice, SLC, Needles Hall, and Roll Up the Rim cups, with seed support for reproducibility.

`WatopolyGame` is the central controller. It implements `IGameContext` and owns the `Board` , the list of `Player` s (as `std::unique_ptr<Player>`), the `Dice` , the `CommandInterpreter` , the `RandomEventGenerator` , the `BoardDisplay` , and a `TurnContext` value struct. After each command, the display is redrawn.

---

3. Class Responsibilities

3.1 Core Engine

- **`IGameContext`** — Abstract interface that all `Square` subclasses and all `Command` subclasses depend on. Provides only the operations that squares and commands need: `sendPlayerToTims` , `triggerAuction` , `promptPurchase` , `rollForGym` (returns a `DiceResult` for gym fee calculation, using `RandomEventGenerator` directly so it does not contaminate `Dice` 's consecutive-doubles tracking), `handleDebt` , `activeCupCount` , `awardCup` , `getCurrentPhase` , `getActivePlayers` , `currentPlayer` , `nextPlayer` , and `findPlayer` . `WatopolyGame` implements this interface. No square or command holds or names the concrete `WatopolyGame` type.

- **`WatopolyGame`** — The top-level controller; implements `IGameContext` . Receives parsed command-line options (testing mode flag, optional seed, optional load file) from `main` , which handles argument parsing. Manages the main game loop: prompts the current player for input, delegates to the `CommandInterpreter` , checks win conditions (last player standing), and triggers display redraws. All per-turn mutable state (current player index, consecutive doubles count, debt creditor, last dice result, current `TurnPhase`) is stored in a `TurnContext` value struct, keeping `WatopolyGame` focused on game-loop orchestration.

- **`TurnContext`** — Value struct that centralises per-turn mutable state: `currentPlayerIndex` , `phase` (`TurnPhase`), `consecutiveDoubles` , `lastRoll` (`DiceResult` — used for the forced Tims Line third-turn exit), and `debtCreditor` (raw non-owning `Player*` , null means debt is owed to the Bank). Held by `WatopolyGame` as a single member; reset at the start of each player's turn.

- **Board** — Owns an array of 40 Square pointers and 10 MonopolyBlock objects. Provides getSquare(int position), positionOf(std::string), and notify(std::vector<Player> const &players) which calls obs->update(*this, players) on each registered observer. Board is a pure container and Observer Subject — it has no knowledge of specific square types or their data. All 40 squares and 10 blocks are constructed by the free function buildDefaultBoard(Board &) (see Section 6), not by Board itself.
- **Player** — Stores: name, display character (PieceType), cash (starts at $1500), board position (0–39), list of owned Property raw non-owning pointers, Roll Up the Rim cup count, DC Tims Line state, and lastRoll (DiceResult). Provides methods for: addCash / deductCash, moveTo(position) / moveBy(steps) (moveBy awards the $200 OSAP bonus only when crossing position 0 — i.e., when new position > 0 after wrap — *not* when landing exactly on position 0, which is handled by CollectOSAP::landOn), addProperty / removeProperty, declareBankruptcy, getTotalWorth, isInTimsLine, enterTimsLine, leaveTimsLine, getLastRoll. Does not own game logic — stores and mutates player state when told to by the game controller.
- **Dice** — Wraps the RandomEventGenerator to produce two die rolls (1–6 each in normal mode, or user-specified values in testing mode). Tracks the number of consecutive doubles rolled by the current player (resets on turn change). Provides roll() returning a DiceResult struct with die1, die2, sum, and isDoubles. In testing mode, roll(int d1, int d2) allows deterministic rolls. Dice is used exclusively for player movement rolls. Gym fee rolls go through IGameContext::rollForGym() using RandomEventGenerator directly, so gym rolls never affect Dice::consecutiveDoubles.

3.2 Squares

- **Square** (abstract) — Base class for all 40 board positions. Provides a name() accessor and a pure virtual landOn(Player&, IGameContext&) method. All squares depend on the IGameContext interface, not on concrete WatopolyGame.
- **Property** (abstract, extends Square) — Base class for ownable squares (academic buildings, residences, gyms). Stores: owner (raw non-owning Player*, or null if unowned), purchaseCost, and isMortgaged flag. Provides: getOwner, setOwner, mortgage (gives owner half of purchase cost, sets mortgaged flag), unmortgage (costs 60% of purchase cost), and a pure virtual calculateFee(Player& visitor, IGameContext& game) method. The landOn implementation in Property determines the landing outcome (unowned / owned-by-self / owned-by-other-mortgaged / owned-by-other-active) and delegates the *decision* to the game: it calls game.promptPurchase(player, *this) for unowned properties (WatopolyGame then handles the buy/auction flow) or game.handleDebt(player, fee, owner) when a fee is owed. Property does not orchestrate auctions or debt resolution directly.
- **AcademicBuilding** (extends Property) — Adds: non-owning MonopolyBlock* reference, improvementCost, numImprovements (0–5), and a tuition table (array of 6 values for 0–5 improvements). calculateFee returns the tuition based on numImprovements; if the owner holds the entire monopoly and numImprovements == 0, the base tuition is doubled. Provides buyImprovement (up to 5, requires owning the full monopoly, costs improvementCost) and sellImprovement (refunds half of improvementCost). Improvements are represented by I characters on the board display.
- **Residence** (extends Property) — Purchase cost is always $200. calculateFee checks how many residences the owner holds (1→$25, 2→$50, 3→$100, 4→$200). No improvements.
- **Gym** (extends Property) — Purchase cost is always $150. calculateFee calls game.rollForGym() to obtain a DiceResult (rolled via RandomEventGenerator, independent of Dice's consecutive-doubles tracking) and multiplies the sum by 4 (if owner has 1 gym) or 10 (if owner has 2 gyms). No improvements.
- **CollectOSAP** — landOn gives the player $200. The passing bonus for crossing position 0 during movement is handled by Player::moveBy, which awards $200 only when the player's path crosses (not lands on) position 0. This prevents double-payment: landing exactly on square 0 triggers CollectOSAP::landOn for the $200 payment, but moveBy does not additionally pay because the final position equals 0 (not a strict crossing).
- **DCTimsLine** — landOn does nothing if the player simply lands on it. The "sent to Tims" logic (from Go to Tims, SLC, or triple doubles) is handled by calling game.sendPlayerToTims(player) via IGameContext. The game controller manages the "leaving Tims" procedure at the start of a jailed player's turn: attempt to roll doubles, pay $50, or use a Roll Up the Rim cup. On the third turn in line, the player must leave; they move the sum stored in TurnContext::lastRoll (the roll from the failed doubles attempt on that turn).
- **GoToTims** — landOn calls game.sendPlayerToTims(player). The player does not collect $200 even if they pass Collect OSAP.
- **GooseNesting** — landOn prints a flavour message ("You have been attacked by a flock of nesting geese!") and does nothing else.
- **TuitionSquare** — landOn prompts the player to choose between paying $300 or 10% of their total worth (cash + printed property values + improvement costs). Payment goes to the Bank. While making this decision, assets and all commands are disabled (enforced via Command::isValidInPhase checking for TurnPhase::TuitionDecision). The 10% helper is a private method (tenPercentOfWorth) — it is not part of the public interface.
- **CoopFee** — landOn immediately deducts $150 from the player. Payment goes to the Bank.
- **SLCSquare** — landOn generates a random event from the SLC probability distribution (see Table 2 in the spec). There is a 1% chance (if fewer than 4 cups are active) of receiving a Roll Up the Rim cup instead of the normal effect; if this triggers, no SLC movement occurs. If the player is moved, the destination is handled as follows: for SLCEvent::GoToTims, game.sendPlayerToTims(player) is called directly (not a recursive landOn on DCTimsLine, which would do nothing). For SLCEvent::AdvanceToOSAP, CollectOSAP::landOn is called recursively (the player does collect $200). For all other move events, the destination square's landOn is called recursively. A message is printed indicating the movement. Note: because the 1% cup check fires *instead of* the SLC table, the effective probability of each SLC event is 0.99 × table probability — an acceptable approximation of the spec's stated distribution.
- **NeedlesHallSquare** — landOn generates a random money change from the Needles Hall probability distribution (see Table 3 in the spec). There is a 1% chance (if fewer than 4 cups are active) of receiving a Roll Up the Rim cup instead. Prints a message indicating the change.

3.3 Ownership & Economy

- **MonopolyBlock** — Groups a set of non-owning AcademicBuilding* pointers that form a monopoly (e.g., Arts1 = {AL, ML}). Owned by Board (as a value in Board's MonopolyBlock array). AcademicBuilding holds a non-owning raw pointer back to its block (stable because Board owns MonopolyBlock for the game's lifetime). Provides isOwnedBy(Player&) (checks if a single player owns all buildings in the block) and hasAnyImprovements() (checks if any building in the block has improvements, relevant for trade restrictions). There are 10 monopoly blocks total.
- **Auction** — Manages a single auction for one property. Takes a list of eligible players (all non-bankrupt players). Each player in turn order may raise the current bid or withdraw. When only one player remains, they pay their final bid and receive the property. Handles edge cases: all players withdraw (property remains unowned), starting bid of $0, etc.
- **Trade** — Encapsulates a trade offer between two players. Stores privately: the offering player, the target player, what is offered (property or cash), and what

is requested (property or cash). Created stack-locally inside `TradeCommand::execute` and destroyed after resolution — `WatopolyGame` does not accumulate `Trade` objects. `validate()` checks: the offer is not money-for-money, the offering player owns the offered property (if applicable), the receiving player owns the requested property (if applicable), and no property in a monopoly with improvements is being traded. If valid, `presentTo(target)` presents the offer for `accept` or `reject`. `execute()` transfers the assets.

3.4 Command Layer

- **`CommandInterpreter`** — Reads input from stdin, parses the command and arguments, and dispatches to the appropriate `Command` object via `IGameContext &`. Handles unknown/misspelled commands gracefully with an error message (no crash). Maintains a registry of valid commands as `std::map<std::string, std::unique_ptr<Command>>` — no raw owning pointers, no explicit `delete`. In testing mode, enables the `roll <die1> <die2>` variant.
- **`Command`** (abstract) — Base class with `execute(IGameContext&, std::vector<std::string> args)` and `isValidInPhase(TurnPhase) const`. Each subclass declares which phases permit its execution; `CommandInterpreter` calls `isValidInPhase` before dispatching. Concrete subclasses:
  - **`RollCommand`** (`roll` / `roll d1 d2`) — Rolls dice, moves player, triggers square `landOn` via `game.resolveLanding`, handles doubles (roll again) and triple doubles (sends to Tims via `game.sendPlayerToTims`). Stores the roll result in `TurnContext::lastRoll`.
  - **`NextCommand`** (`next`) — Calls `game.nextPlayer()`. Only valid when the player cannot roll.
  - **`TradeCommand`** (`trade <name> <give> <receive>`) — Creates a stack-local `Trade`, validates, and processes it.
  - **`ImproveCommand`** (`improve <property> buy/sell`) — Buys or sells an improvement on the specified academic building, with monopoly and improvement-limit validation.
  - **`MortgageCommand`** (`mortgage <property>`) — Mortgages the specified property, validating ownership and no-improvement constraints.
  - **`UnmortgageCommand`** (`unmortgage <property>`) — Unmortgages the specified property and charges 60% of purchase cost (or 70% if the property was received via bankruptcy transfer — see Section 5.3).
  - **`BankruptCommand`** (`bankrupt`) — Declares bankruptcy when the player owes more than available cash; transfers/auctions assets based on creditor. Reads `TurnContext::debtCreditor`.
  - **`AssetsCommand`** (`assets`) — Displays the current player's assets. Not valid during `TuitionDecision` phase.
  - **`AllCommand`** (`all`) — Displays every player's assets. Not valid during `TuitionDecision` phase.
  - **`SaveCommand`** (`save <filename>`) — Writes the full game state to file in the required save format.

3.5 Persistence

- **`SaveManager`** — Handles reading and writing game state files. `save(WatopolyGame&, filename)` serializes: number of players, each player's data (name, char, tims cups, money, position, DC Tims Line state), and each ownable property's data (owner, improvements/mortgage status) in board order. `load(filename)` reconstructs the full game state from a file, creating players and assigning property ownership/improvements accordingly. Validates: player count (2–6), no player on square 30, DC Tims Line encoding correctness, owner name is a known player or `BANK`, improvement counts in range, total cups ≤ 4.

3.6 Presentation

- **`DisplayObserver`** — Interface with `update(Board const &board, std::vector<Player> const &players)`. Allows the game to trigger redraws without knowing the concrete display type. Board calls `notify(players)` which iterates its observer list and calls `update(*this, players)` on each — Board receives the player vector from WatopolyGame at notify time, satisfying the update signature without Board needing to own players.
- **`BoardDisplay`** (extends `DisplayObserver`) — Renders the text-based board to stdout. Reads the current state of all 40 squares and all players to produce the grid layout shown in the spec (with player characters on their current square and improvement `I` markers on academic buildings). The board is redrawn after every command.

3.7 Randomness

- **`RandomEventGenerator`** — Wraps `std::default_random_engine`. Seeded once at startup (from command-line argument or system clock). Provides methods: `rollDie()` (1–6), `generateSLCEvent()` (returns movement offset or special action per Table 2 probabilities), `generateNeedlesHallDelta()` (returns money change per Table 3 probabilities), `rollUpTheRimCheck()` (returns true with 1% probability, subject to the 4-cup global cap). `IGameContext::rollForGym()` calls `rollDie()` twice through this class directly (bypassing `Dice`), ensuring gym rolls never affect the consecutive-doubles counter. Centralizing all RNG ensures reproducibility when a seed is provided.

---

4. Board Layout & Square Ordering

The 40 squares are arranged clockwise starting from Collect OSAP:

- `0`: Collect OSAP (Non-property)
- `1`: AL (Academic Building - Arts1)
- `2`: SLC (Non-property)
- `3`: ML (Academic Building - Arts1)
- `4`: Tuition (Non-property)
- `5`: MKV (Residence)
- `6`: ECH (Academic Building - Arts2)
- `7`: Needles Hall (Non-property)
- `8`: PAS (Academic Building - Arts2)
- `9`: HH (Academic Building - Arts2)
- `10`: DC Tims Line (Non-property)
- `11`: RCH (Academic Building - Eng)
- `12`: PAC (Gym)

- `13` : DWE (Academic Building - Eng)
- `14` : CPH (Academic Building - Eng)
- `15` : UWP (Residence)
- `16` : LHI (Academic Building - Health)
- `17` : SLC (Non-property)
- `18` : BMH (Academic Building - Health)
- `19` : OPT (Academic Building - Health)
- `20` : Goose Nesting (Non-property)
- `21` : EV1 (Academic Building - Env)
- `22` : Needles Hall (Non-property)
- `23` : EV2 (Academic Building - Env)
- `24` : EV3 (Academic Building - Env)
- `25` : V1 (Residence)
- `26` : PHYS (Academic Building - Sci1)
- `27` : B1 (Academic Building - Sci1)
- `28` : CIF (Gym)
- `29` : B2 (Academic Building - Sci1)
- `30` : Go to Tims (Non-property)
- `31` : EIT (Academic Building - Sci2)
- `32` : ESC (Academic Building - Sci2)
- `33` : Needles Hall (Non-property)
- `34` : C2 (Academic Building - Sci2)
- `35` : REV (Residence)
- `36` : Needles Hall (Non-property)
- `37` : MC (Academic Building - Math)
- `38` : Coop Fee (Non-property)
- `39` : DC (Academic Building - Math)

---

5. Game Flow Overview

5.1 Startup

1. `main` parses command-line arguments: `-load <file>`, `-testing`, and optionally `-seed <n>`, then constructs `WatopolyGame` with the parsed options.
2. If `-load` is specified, `SaveManager::load` reconstructs the game state from file. Otherwise, the game prompts for the number of players (2–6), each player's name, and their chosen display character. Token uniqueness is enforced (no two players may share a token), and the name `BANK` is rejected (it is reserved as the unowned-property sentinel in save files).
3. `buildDefaultBoard(board)` (a free function) initializes all 40 squares with their correct parameters and all 10 `MonopolyBlock` objects. `AcademicBuilding` objects receive a non-owning pointer to their `MonopolyBlock`; `MonopolyBlock` objects receive non-owning pointers back to their buildings. Both squares and blocks are owned by `Board`.
4. Players are stored as `std::vector<std::unique_ptr<Player>>` so that raw `Player*` back-pointers in `Property::owner` remain stable across any vector operations (no reallocation invalidates heap addresses).
5. The `RandomEventGenerator` is seeded (from `-seed` argument or system clock).
6. The `BoardDisplay` is registered as an observer on the `Board` and an initial board is drawn.

5.2 Turn Loop

1. The game announces whose turn it is. `TurnContext` is reset for the new player.
2. **Pre-roll phase** (`TurnPhase::PreRoll`): The player may issue non-roll commands (`trade`, `improve`, `mortgage`, `unmortgage`, `assets`, `all`, `save`) at any time.
3. **Roll phase:**
   - If the player is **in the DC Tims Line**, they follow the leaving procedure:
     - Option A: Try to roll doubles. If successful, move the rolled sum and leave the line. The roll is stored in `TurnContext::lastRoll`.
     - Option B: Pay $50 to leave, then roll and move normally.
     - Option C: Use a Roll Up the Rim cup to leave, then move using `TurnContext::lastRoll` (the sum from the failed doubles attempt on this same turn, or a new roll if they paid/used a cup before rolling).
     - On the third turn in line without rolling doubles, the player must pay $50 or use a cup, then move the sum from `TurnContext::lastRoll`.
   - If the player is **not in the DC Tims Line**, they roll two dice. Their token moves the sum. `landOn` is called on the destination square via the square's `IGameContext` dependency.
     - If they rolled doubles, they roll again (returning to step 3).
     - If they roll three consecutive doubles, `game.sendPlayerToTims(player)` is called instead of moving.
   - Movement past position 0 (Collect OSAP) awards $200 via `Player::moveBy` only when strictly crossing (new position > 0). Landing exactly on position 0 triggers `CollectOSAP::landOn` for the single $200 award.
4. **Post-roll phase** (`TurnPhase::PostRoll`): The player may continue issuing non-roll commands. When done, they enter `next` to pass control to the next player.
5. The board is redrawn after every command.

5.3 Owing Money

When a player must pay more than they currently have, `TurnContext::debtCreditor` is set (non-null for a player creditor, null for Bank) and `TurnContext::phase` transitions to `TurnPhase::DebtResolution`:

1. The player is given the opportunity to raise funds by selling improvements (`improve <prop> sell`), mortgaging properties (`mortgage <prop>`), or trading.
2. If the player raises enough money, the debt is paid and play continues.

3. If the player cannot or chooses not to raise enough money, they may declare `bankrupt`.
    - **Bankruptcy to another player:** All of the bankrupt player's assets (cash, properties, Roll Up the Rim cups) are transferred to the creditor. For each mortgaged property transferred, `0.1 × purchaseCost` is immediately deducted from the creditor and paid to the Bank. The creditor may then choose to unmortgage the property now (paying the principal: 50% of purchase cost) or leave it mortgaged. If they leave it mortgaged and later unmortgage it, they pay an additional 10% on top of the normal 60% cost (total 70%) as specified by the spec.
    - **Bankruptcy to the Bank:** All properties are returned to the market as unmortgaged and auctioned off individually. All Roll Up the Rim cups are destroyed.
4. The bankrupt player is removed from the game. Player storage as `std::vector<std::unique_ptr<Player>>` ensures no other `Player*` pointers are invalidated by the removal.

5.4 Auctions

1. Triggered when a player declines to buy a property they landed on (WatopolyGame receives the outcome from `promptPurchase` and calls `triggerAuction`), or when a player goes bankrupt to the Bank.
2. All remaining (non-bankrupt) players participate.
3. In turn order, each player may raise the bid or withdraw.
4. The last remaining bidder wins and pays their final bid for the property.

5.5 End Condition

The game ends when only one player remains (all others have declared bankruptcy). That player is declared the winner.

---

6. Design Patterns & Rationale

Observer Pattern — Board Display

The `Board` acts as the Subject. `BoardDisplay` (and any future display types) registers as an Observer. After any state change, `WatopolyGame` calls `board.notify(players)`, which passes both the Board state and the player list to `update(board, players)` on each observer. This decouples game logic from presentation entirely. Adding a graphical or network display means creating a new `DisplayObserver` subclass and registering it — no changes to game logic.

Dependency Inversion Principle — IGameContext

The `IGameContext` abstract interface is the primary architectural tool for decoupling. All `Square` subclasses and all `Command` subclasses depend on `IGameContext &`, never on the concrete `WatopolyGame`. This means none of the 11 square types or 10 command types need to be recompiled when `WatopolyGame`'s internal implementation changes. It also makes unit testing feasible: a mock `IGameContext` can be injected without instantiating the full game.

Template Method Pattern — Property Purchase Flow

The `Property` base class implements the common `landOn` flow as a template method. It determines the landing outcome (unowned, self-owned, other-owned) and delegates to the game via `IGameContext`. Subclasses override `calculateFee` to provide type-specific fee computation (tuition tables for academic buildings, residence count for residences, dice-based for gyms). This avoids duplicating the purchase/ownership logic across three property types.

Factory Function — Board Initialization

The free function `buildDefaultBoard(Board &board)` constructs all 40 `Square` objects and all 10 `MonopolyBlock` objects with their correct parameters and places them in the `Board`. This separates data-heavy initialization from the `Board` container class, which is a pure holder and Observer Subject. Changing board data (e.g., tuition rebalance) or adding a new square type requires only changes to `buildDefaultBoard`, not to `Board` itself.

Polymorphism / Open-Closed Principle — Square Landing Behaviour

Each `Square` subclass implements its own `landOn` method, encapsulating the behaviour specific to that square type. The game controller calls `square->landOn(player, game)` without needing to know the concrete type. Adding a new square type means creating a new subclass — no modification to the game controller or other squares. This is the Open-Closed Principle enabled by runtime polymorphism; it is not the Strategy Pattern (which requires an algorithm object swappable at runtime on a host).

---

7. Answers to Project Questions

Q1: After reading the Buildings subsection, would the Observer Pattern be a good pattern to use when implementing a game board? Why or why not?

Answer: Yes, the Observer Pattern is a strong fit for the game board. The board's visual representation needs to update whenever the game state changes — players move, properties change ownership, improvements are built or sold, players go bankrupt, etc. Rather than having every piece of game logic explicitly call a "redraw" function (which would create tight coupling between game logic and display code), we make the `Board` a Subject that notifies registered `DisplayObserver`s after each state change.

This provides several concrete benefits:

1. **Decoupling:** Game logic classes (`WatopolyGame`, `Player`, `Square` subclasses) do not need to know about the display at all. They modify state, and the Observer mechanism handles the rest.

2. **Extensibility:** If we later wanted to add a graphical display, a logging observer, or a network observer (for remote play), we simply create a new `DisplayObserver` subclass and register it — no changes to the game logic.
3. **Single responsibility:** The `Board` class focuses on maintaining square data. The `BoardDisplay` class focuses solely on rendering. Neither has to understand the other's internals.

The main consideration is performance: redrawing a text board after every single command could produce a lot of output. However, since this is a turn-based game with human-speed input, this is not a practical concern.

Q2: Suppose that we wanted to model SLC and Needles Hall more closely to Chance and Community Chest cards. Is there a suitable design pattern you could use? How would you use it?

Answer: The **Command Pattern** (or equivalently the **Strategy Pattern** applied to card effects) would be well-suited for this.

Instead of hardcoding the probability distributions and effects directly inside `SLCSquare::landOn` and `NeedlesHallSquare::landOn`, we would model each possible outcome as its own class implementing a common `Card` interface:

```
class Card {
public:
    virtual void execute(Player& player, IGameContext& game) = 0;
    virtual std::string description() const = 0;
    virtual ~Card() = default;
};
```

Concrete cards would include:

- `MoveForwardCard(int n)` — moves the player forward by `n` squares.
- `MoveBackwardCard(int n)` — moves the player backward by `n` squares.
- `GoToTimsCard` — calls `game.sendPlayerToTims(player)`.
- `AdvanceToOSAPCard` — moves the player to Collect OSAP (collecting $200).
- `GainMoneyCard(int amount)` — adds money to the player.
- `LoseMoneyCard(int amount)` — deducts money from the player.
- `RollUpTheRimCard` — calls `game.awardCup(player)`.

A `CardDeck` class would hold a shuffled collection of `Card` objects (with quantities matching the specified probability distributions). `SLCSquare` and `NeedlesHallSquare` would each own a `CardDeck` and, on `landOn`, draw the top card and call `card->execute(player, game)`.

This approach has several advantages:

- **Adding new effects** requires only a new `Card` subclass and adding it to the deck — no modification to `SLCSquare` or `NeedlesHallSquare`.
- **Changing probabilities** is a matter of adjusting how many of each card type are in the deck.
- **Deck behaviour** (e.g., draw-without-replacement and reshuffle when empty, like real Monopoly) can be encapsulated in `CardDeck`.
- The pattern mirrors the physical card decks in actual Monopoly, making the code intuitive.

Q3: Is the Decorator Pattern a good pattern to use when implementing Improvements? Why or why not?

Answer: The Decorator Pattern is **not** a good fit for implementing improvements in Watopoly. While it might seem appealing at first glance (each improvement "wraps" the building and modifies its tuition), there are several reasons why a simpler design is preferable:

1. **Improvements are uniform and countable.** Each academic building has exactly 6 tuition tiers (0–5 improvements), and the tuition for each tier is a fixed value from a lookup table. The Decorator Pattern is most useful when decorators add diverse, composable behaviours. Here, each "decorator" would do the same thing — change the tuition to the next value in the table — making the wrapping overhead pointless.

2. **Selling improvements requires unwrapping.** Improvements can be sold in reverse order. With the Decorator Pattern, selling the most recent improvement means removing the outermost wrapper, which is awkward in a linked chain of decorators. With a simple integer counter, selling is just `numImprovements--`.

3. **Querying the improvement count must be easy.** The board display, save/load, trade validation, and mortgage validation all need to quickly check how many improvements a building has. With decorators, this requires traversing the chain. With an integer field, it is `O(1)`.

4. **No combinatorial variety.** In Watopoly, the 5 improvements are sequential. There is no scenario where improvements from different "types" are mixed. The Decorator Pattern shines when decorations can be composed in arbitrary combinations; that flexibility is not needed here.

Our approach: Each `AcademicBuilding` stores an `int numImprovements` (0–5) and a tuition lookup table (array of 6 values). `calculateFee` simply indexes into the table. `buyImprovement` increments the counter (after validating monopoly ownership and the max of 5). `sellImprovement` decrements it and refunds half the improvement cost. This is simple, efficient, and easy to understand.

---

8. Key Design Decisions & Resilience to Change

8.1 Adding New Square Types

All squares inherit from the `Square` abstract class and implement `landOn(Player&, IGameContext&)`. To add a new square type (e.g., a "Scholarship" square that gives a random reward), one creates a new subclass, implements `landOn`, and adds it to `buildDefaultBoard`. No existing classes need modification.

8.2 Adding New Commands

Each command is a subclass of `Command` with an `execute(IGameContext&, args)` method and an `isValidInPhase` implementation. The

`CommandInterpreter` maintains a registry (using `std::unique_ptr<Command>` values) mapping command strings to `Command` objects. To add a new command, create the subclass and register it — no modification to the interpreter's parsing logic or other commands.

8.3 Changing Tuition/Rent/Fee Calculations

All fee calculations are encapsulated in the respective `Property` subclass's `calculateFee` method. Tuition values are stored in data tables, not hardcoded into control flow. Changing a value means updating the table; changing the formula means modifying one method in one class.

8.4 Changing the Number of Players

The game supports 2–6 players via a dynamic list of `Player` objects stored as `std::vector<std::unique_ptr<Player>>`. Turn order iterates through this list and skips bankrupt players. No part of the code assumes a fixed number of players.

8.5 Adding Graphics Display

Because the display uses the Observer Pattern, adding a graphical display means creating a new `DisplayObserver` subclass (e.g., `GraphicsDisplay`) and registering it alongside `BoardDisplay`. The game logic is completely unaware of how many or what kind of observers exist.

8.6 Changing Board Layout

The board is initialized by `buildDefaultBoard`, a free function that creates all 40 squares and places them in an array. Rearranging squares, adding new ones, or removing existing ones requires changing only this function. The rest of the game operates on `Square` pointers by position index and does not assume any particular ordering.

8.7 Changing SLC / Needles Hall Probabilities

The probability distributions are stored as data in `RandomEventGenerator`. Changing probabilities is a data change, not a logic change. If we adopt the Strategy/Card pattern described in Q2, it becomes even simpler — just adjust the card counts in the deck.

8.8 Changing the Save/Load Format

All serialization logic is contained within `SaveManager`. If the file format changes, only `SaveManager` needs to be updated. The game's internal representation is not coupled to the file format.

8.9 Changing Game Rules That Affect Multiple Square Types or Commands

Because all square and command logic goes through the `IGameContext` interface, changing a game rule that is brokered by the game engine (e.g., "gyms now charge 5× instead of 4×", or "players no longer collect $200 when passing Go") requires changing only `WatopolyGame`'s implementation of the relevant `IGameContext` method. No `Square` or `Command` subclass needs to be modified. The interface acts as a firewall between game-rule changes and the classes that trigger them.

---

9. Implementation Roadmap

1. **Day 1**
    - **Cristophe Chen:** Finalize UML draft + class relationships for `WatopolyGame`, `IGameContext`, `Board`, `TurnContext`, and `Player`.
    - **Brian Fu:** Finalize UML draft + command layer classes (`CommandInterpreter`, `Command` hierarchy).
    - **Kevin Wang:** Finalize UML draft + square/property hierarchy (`Square`, `Property`, specialized squares).
2. **Day 2**
    - **Cristophe Chen:** Implement project skeleton (`order.txt`, module files, shared enums/structs including `DiceResult`, `TurnContext`, `TurnPhase`).
    - **Brian Fu:** Implement `Square` base + non-property squares (`CollectOSAP`, `GooseNesting`, `CoopFee`).
    - **Kevin Wang:** Implement remaining non-property squares (`DCTimsLine`, `GoToTims`, `SLC`, `NeedlesHall`, `Tuition`).
3. **Day 3**
    - **Cristophe Chen:** Implement `Property` base class + `MonopolyBlock` + `buildDefaultBoard` free function.
    - **Brian Fu:** Implement `AcademicBuilding` (tuition table, improvement buy/sell rules).
    - **Kevin Wang:** Implement `Residence` + `Gym` fee logic and ownership hooks.
4. **Day 4**
    - **Cristophe Chen:** Implement `WatopolyGame` core + `IGameContext` implementation + `TurnContext` + doubles/triple-doubles logic.
    - **Brian Fu:** Implement `CommandInterpreter` skeleton (`roll`, `next`, parser validation, `isValidInPhase` gating).
    - **Kevin Wang:** Implement `Board` + `buildDefaultBoard` data + `BoardDisplay` text rendering hookup.
5. **Day 5**
    - **Cristophe Chen:** Implement purchase flow + `Auction` bidding loop.
    - **Brian Fu:** Implement economy commands (`improve`, `mortgage`, `unmortgage`) with rule checks.
    - **Kevin Wang:** Integrate movement + landing effects + OSAP pass/land handling (single-payment logic in `moveBy`).
6. **Day 6**

- **Cristophe Chen:** Implement trading system (`trade` , offer validation, accept/reject flow, stack-local `Trade`).
- **Brian Fu:** Implement bankruptcy flow to player/Bank, including mortgaged transfer 10% fee and auctions.
- **Kevin Wang:** Implement DC Tims Line complete flow (roll/pay/cup/third-turn forced exit using `TurnContext::lastRoll`).

7. **Day 7**

   - **Cristophe Chen:** Implement `SaveManager::save` and serialization format validation.
   - **Brian Fu:** Implement `SaveManager::load` and full state reconstruction (`-load` startup path).
   - **Kevin Wang:** Implement testing mode (`-testing` , `roll <die1> <die2>`) and RNG seed option.

8. **Day 8**

   - **Cristophe Chen:** Write integration tests for core game loop (movement, doubles, turns, win condition).
   - **Brian Fu:** Write integration tests for economy (purchase, rent, improve, mortgage, bankruptcy).
   - **Kevin Wang:** Write integration tests for special squares (SLC, Needles Hall, Tims Line, Tuition, cups).

9. **Day 9**

   - **Cristophe Chen:** Final integration pass + edge-case fixes.
   - **Brian Fu:** Final integration pass + command robustness (invalid input and recovery).
   - **Kevin Wang:** Final integration pass + display/save-load consistency checks.

10. **Day 10**

    - **Cristophe Chen:** Final checks + Marmoset dry run and submission prep.
    - **Brian Fu:** Final checks + Marmoset dry run and submission prep.
    - **Kevin Wang:** Final checks + Marmoset dry run and submission prep.